

utility-functions-in-pyspark

January 22, 2024

1 Data Manipulations in PySpark

```
[1]: # Import Sparksession
from pyspark.sql import SparkSession
spark=SparkSession.builder.appName("Data_Wrangling").getOrCreate()
```

```
[2]: # Print PySpark and Python versions
import sys
print('Python version: '+sys.version)
print('Spark version: '+spark.version)
```

```
Python version: 3.8.3 | packaged by conda-forge | (default, Jun 1 2020,
17:43:00)
[GCC 7.5.0]
Spark version: 3.0.0
```

```
[3]: # Read data
file_location = "movie_data_part1.csv"
file_type = "csv"
infer_schema = "false"
first_row_is_header = "true"
delimiter = "|"

df = spark.read.format(file_type)\
.option("inferSchema", infer_schema)\
.option("header", first_row_is_header)\
.option("sep", delimiter)\
.load(file_location)
```

```
[4]: # Print Metadata
df.printSchema()
```

```
root
 |-- belongs_to_collection: string (nullable = true)
 |-- budget: string (nullable = true)
 |-- id: string (nullable = true)
 |-- original_language: string (nullable = true)
 |-- original_title: string (nullable = true)
```

```

|-- overview: string (nullable = true)
|-- popularity: string (nullable = true)
|-- production_companies: string (nullable = true)
|-- production_countries: string (nullable = true)
|-- release_date: string (nullable = true)
|-- revenue: string (nullable = true)
|-- runtime: string (nullable = true)
|-- status: string (nullable = true)
|-- tagline: string (nullable = true)
|-- title: string (nullable = true)
|-- vote_average: string (nullable = true)

```

```

[5]: # Print Metadata - 2nd method
df.dtypes

```

```

[5]: [('belongs_to_collection', 'string'),
      ('budget', 'string'),
      ('id', 'string'),
      ('original_language', 'string'),
      ('original_title', 'string'),
      ('overview', 'string'),
      ('popularity', 'string'),
      ('production_companies', 'string'),
      ('production_countries', 'string'),
      ('release_date', 'string'),
      ('revenue', 'string'),
      ('runtime', 'string'),
      ('status', 'string'),
      ('tagline', 'string'),
      ('title', 'string'),
      ('vote_average', 'string')]

```

```

[46]: # String functions
df_with_newcols = df.select('id','budget','popularity').\
withColumn('budget_cat', when(df['budget']<10000000,'Small').\
    ↪when(df['budget']<100000000,'Medium').otherwise('Big')).\
withColumn('ratings', when(df['popularity']<3,'Low').\
    ↪when(df['popularity']<5,'Mid').otherwise('High'))

df_with_newcols=df_with_newcols.
    ↪withColumn('BudgetRating_Category',concat(df_with_newcols.
    ↪budget_cat,df_with_newcols.ratings))
df_with_newcols=df_with_newcols.
    ↪withColumn('BudgetRating_Category',trim(lower(df_with_newcols.
    ↪BudgetRating_Category)))
df_with_newcols.show()

```

id	budget	popularity	budget_cat	ratings	BudgetRating_Category
43000	0.0	2.503	Small	Low	smalllow
43001	0.0	5.51	Small	High	smallhigh
43002	0.0	5.62	Small	High	smallhigh
43003	0.0	7.159	Small	High	smallhigh
43004	500000.0	3.988	Small	Mid	smallmid
43006	0.0	3.194	Small	Mid	smallmid
43007	0.0	2.689	Small	Low	smalllow
43008	0.0	6.537	Small	High	smallhigh
43010	0.0	4.297	Small	Mid	smallmid
43011	0.0	4.417	Small	Mid	smallmid
43012	7000000.0	4.722	Small	Mid	smallmid
43013	0.0	2.543	Small	Low	smalllow
43014	0.0	4.303	Small	Mid	smallmid
43015	0.0	3.493	Small	Mid	smallmid
43016	0.0	2.851	Small	Low	smalllow
43017	0.0	4.047	Small	Mid	smallmid
43018	0.0	2.661	Small	Low	smalllow
43019	0.0	3.225	Small	Mid	smallmid
43020	0.0	5.72	Small	High	smallhigh
43021	0.0	3.292	Small	Mid	smallmid

only showing top 20 rows

```
[47]: # Registering temporary table
df_with_newcols.registerTempTable('temp_data')

# Showing the results
spark.sql('select ratings, count(ratings) as ratings_count from temp_data group
↳by ratings').show()
```

ratings	ratings_count
High	16856
Low	14865
Mid	12277

```
[48]: # Deciling
from pyspark.sql.window import *
df_with_newcols=df_with_newcols.filter( (df_with_newcols['popularity'].
↳isNull()) & (~isnan(df_with_newcols['popularity'])) )
```

```
df_with_newcols = df_with_newcols.select("id", "budget", "popularity", ntile(10).
    ↳over(Window.partitionBy().orderBy(df_with_newcols['popularity'].desc()))).
    ↳alias("decile_rank"))
df_with_newcols.groupby("decile_rank").agg(min('popularity').
    ↳alias('min_popularity'), max('popularity').
    ↳alias('max_popularity'), count('popularity')).show()
```

```
+-----+-----+-----+-----+
|decile_rank|min_popularity|max_popularity|count(popularity)|
+-----+-----+-----+-----+
|          1|          10.185|          180.0|          4379|
|          2|           7.481|          10.182|          4379|
|          3|           5.841|           7.481|          4379|
|          4|           4.823|           5.841|          4378|
|          5|           4.054|           4.822|          4378|
|          6|           3.383|           4.054|          4378|
|          7|           2.747|           3.383|          4378|
|          8|           2.075|           2.747|          4378|
|          9|           1.389|           2.075|          4378|
|         10|            0.6|           1.389|          4378|
+-----+-----+-----+-----+
```

```
[49]: # Data prep for windowing function
df_second_best = df.select('id', 'popularity', 'release_date')
df_second_best=df_second_best.withColumn('release_year',year('release_date')).
    ↳drop('release_date')
```

```
[50]: # Defining partition function
year_window = Window.partitionBy(df_second_best['release_year']).
    ↳orderBy(df_with_newcols['popularity'].desc())
df_second_best=df_second_best.select('id', 'popularity', 'release_year',rank().
    ↳over(year_window).alias("rank"))
```

```
[51]: df_second_best.filter((df_second_best['release_year']==1970) &_
    ↳(df_second_best['rank']==2)).show()
```

```
+-----+-----+-----+-----+
| id|popularity|release_year|rank|
+-----+-----+-----+-----+
|11202|    14.029|        1970|   2|
+-----+-----+-----+-----+
```

```
[52]: # Revenue Difference

df_revenue = df.select('id', 'revenue', 'release_date')
```

```
df_revenue=df_revenue.withColumn('release_year',year('release_date')).
↳drop('release_date')

windowRev =Window.partitionBy(df_revenue['release_year']).
↳orderBy(df_revenue['revenue'].desc()).rangeBetween(-sys.maxsize, sys.maxsize)

revenue_difference = (max(df_revenue['revenue']).over(windowRev) -
↳df_revenue['revenue'])
df_revenue.select('id', 'revenue', 'release_year',revenue_difference.
↳alias("revenue_difference")).show(10,False)
```

```
+-----+-----+-----+-----+
|id      |revenue |release_year|revenue_difference|
+-----+-----+-----+-----+
|665     |1.64E8  |1959        |0.0                |
|10882   |5.1E7   |1959        |1.13E8             |
|239     |2.5E7   |1959        |1.39E8             |
|4952    |1.875E7 |1959        |1.4525E8           |
|15944   |1.7658E7|1959        |1.46342E8          |
|213     |1.3275E7|1959        |1.50724992E8       |
|27029   |1.28E7  |1959        |1.512E8            |
|29996   |1.22E7  |1959        |1.518E8            |
|11571   |1.0E7   |1959        |1.54E8             |
|9660    |6800000.0|1959       |1.572E8            |
+-----+-----+-----+-----+
only showing top 10 rows
```

```
[53]: # Collect list demonstration
# Step 1:Create the year column from release date
df=df.withColumn('release_year',year('release_date'))
# Applying collect list function to gather all occurrences
df.filter("title='The Lost World'").groupby('title').
↳agg(collect_list("release_year")).show(1,False)
```

```
+-----+-----+
|title      |collect_list(release_year)|
+-----+-----+
|The Lost World|[1999, 2001, 1925, 1960, 1992, 1998]|
+-----+-----+
```

```
[54]: # Simple random sampling in pyspark without replacement

df_sample = df.sample(False, 0.4, 42)
df_sample.count()
```

[54]: 17711

```
[55]: # Simple random sampling in pyspark with replacement
```

```
df_sample = df.sample(True, 0.4, 42)
df_sample.count()
```

[55]: 17745

```
[56]: # Stratified sampling in pyspark
```

```
df_strat = df.sampleBy("release_year", fractions={1959: 0.2, 1960: 0.4, 1961: 0.4}, seed=11)
df_strat.count()
```

[56]: 260

```
[57]: df.show()
```

```
+-----+-----+-----+-----+-----+-----+-----+
| id| budget|popularity|release_date| revenue| title|
mean_popularity|          varaiance|release_year|
+-----+-----+-----+-----+-----+-----+-----+
|43000|    0.0|    2.503| 1962-05-23|    0.0|The Elusive
Corporal|5.295444259579189|  7.797744825681142|    1962|
|43001|    0.0|    5.51| 1962-11-12|    0.0| Sundays and
Cybele|5.295444259579189|0.046034263963398346|    1962|
|43002|    0.0|    5.62| 1962-05-24|    0.0|Lonely Are the
Brave|5.295444259579189| 0.10533635435520017|    1962|
|43003|    0.0|    7.159| 1975-03-12|    0.0|          F for
Fake|5.295444259579189|  3.4728396990815216|    1975|
|43004| 500000.0|    3.988| 1962-10-09|    0.0|Long Day's
Journe...|5.295444259579189|  1.709410711356759|    1962|
|43006|    0.0|    3.194| 1962-03-09|    0.0|          My
Geisha|5.295444259579189|  4.416067952069205|    1962|
|43007|    0.0|    2.689| 1962-10-31|    0.0|Period of
Adjustment|5.295444259579189|  6.793552245032679|    1962|
|43008|    0.0|    6.537| 1959-03-13|    0.0|    The Hanging
Tree|5.295444259579189|  1.5414611017708733|    1959|
|43010|    0.0|    4.297| 1962-01-01|    0.0|Sherlock Holmes
a...|5.295444259579189|  0.996891076602093|    1962|
|43011|    0.0|    4.417| 1962-01-01|    0.0| Sodom and
Gomorra|5.295444259579189| 0.7716646388835591|    1962|
|43012|7000000.0|    4.722| 1962-11-21|4000000.0|          Taras
Bulba|5.295444259579189| 0.32883817884329936|    1962|
|43013|    0.0|    2.543| 1962-04-17|    0.0|The Counterfeit
```

```

T...|5.295444259579189| 7.575949496588105| 1962|
|43014| 0.0| 4.303| 1962-10-24| 0.0| Tower of
London|5.295444259579189| 0.9849456613739271| 1962|
|43015| 0.0| 3.493| 1962-12-07| 0.0|Varan the
Unbelie...|5.295444259579189| 3.248805198877505| 1962|
|43016| 0.0| 2.851| 1962-01-01| 0.0|Waltz of the
Tore...|5.295444259579189| 5.975307393171506| 1962|
|43017| 0.0| 4.047| 1961-10-11| 0.0| Back
Street|5.295444259579189| 1.5586132407239628| 1961|
|43018| 0.0| 2.661| 1961-06-02| 0.0|Gidget Goes
Hawaiian|5.295444259579189| 6.940296486482491| 1961|
|43019| 0.0| 3.225| 2010-05-28| 0.0|Schuks
Tshabalala...|5.295444259579189| 4.2867398269303285| 2010|
|43020| 0.0| 5.72| 1961-06-15| 0.0|The Colossus of
R...|5.295444259579189| 0.1802473985736286| 1961|
|43021| 0.0| 3.292| 2008-08-22| 0.0| Sex
Galaxy|5.295444259579189| 4.013788679607341| 2008|
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+
only showing top 20 rows

```

```

[58]: # Coalesce Demonstration
df.coalesce(1).write.format('csv').option('delimiter', '|').save('output_df1')

```

```

[59]: # PartitionBy Demonstration
df.write.partitionBy('release_year').format('csv').option('delimiter', '|').
    ↪save('output_df_partitioned')

```

```

[60]: # SaveAs Demonstration
df.write.saveAsTable('film_ratings')

```

```

[61]: # Pyspark to Pandas
df_pandas=df.toPandas()

```

```

[62]: # Pandas to PySpark
df_py = spark.createDataFrame(df_pandas)

```

```

[63]: # Reading another new dataset for joins

# Input parameters
file_location = "movie_data_part2.csv"
file_type = "csv"
infer_schema = "False"
first_row_is_header = "True"
delimiter = "|"

```

```
# Bringing all the options together to read the csv file
```

```
df_p1 = spark.read.format(file_type) \  
  .option("inferSchema", infer_schema) \  
  .option("header", first_row_is_header) \  
  .option("sep", delimiter) \  
  .load(file_location)
```

```
[64]: # Schema of new dataset  
df.printSchema()
```

```
root  
 |-- id: integer (nullable = true)  
 |-- budget: float (nullable = true)  
 |-- popularity: float (nullable = true)  
 |-- release_date: date (nullable = true)  
 |-- revenue: float (nullable = true)  
 |-- title: string (nullable = true)  
 |-- mean_popularity: double (nullable = false)  
 |-- varaiance: double (nullable = true)  
 |-- release_year: integer (nullable = true)
```

```
[65]: # Preparing data for joins  
df_p1 = df_p1.withColumn('id',df_p1['id'].cast("integer"))  
df_p1=df_p1.filter((df_p1['id'].isNotNull()) & (~isnan(df_p1['id'])))
```

```
[66]: # Inner Join  
df.join(df_p1, df['id'] == df_p1['id'],'inner').printSchema()  
df.join(df_p1, df['id'] == df_p1['id']).count()
```

```
root  
 |-- id: integer (nullable = true)  
 |-- budget: float (nullable = true)  
 |-- popularity: float (nullable = true)  
 |-- release_date: date (nullable = true)  
 |-- revenue: float (nullable = true)  
 |-- title: string (nullable = true)  
 |-- mean_popularity: double (nullable = false)  
 |-- varaiance: double (nullable = true)  
 |-- release_year: integer (nullable = true)  
 |-- cast: string (nullable = true)  
 |-- adult: string (nullable = true)  
 |-- directors: string (nullable = true)  
 |-- vote_count: string (nullable = true)  
 |-- spoken_languages: string (nullable = true)  
 |-- poster_path: string (nullable = true)
```

```
|-- homepage: string (nullable = true)
|-- imdb_id: string (nullable = true)
|-- genres: string (nullable = true)
|-- video: string (nullable = true)
|-- id: integer (nullable = true)
```

[66]: 43783

```
[67]: # Left Join
df.join(df_p1, df['id'] == df_p1['id'], 'left').printSchema()
df.join(df_p1, df['id'] == df_p1['id'], 'left').count()
```

```
root
|-- id: integer (nullable = true)
|-- budget: float (nullable = true)
|-- popularity: float (nullable = true)
|-- release_date: date (nullable = true)
|-- revenue: float (nullable = true)
|-- title: string (nullable = true)
|-- mean_popularity: double (nullable = false)
|-- varaiance: double (nullable = true)
|-- release_year: integer (nullable = true)
|-- cast: string (nullable = true)
|-- adult: string (nullable = true)
|-- directors: string (nullable = true)
|-- vote_count: string (nullable = true)
|-- spoken_languages: string (nullable = true)
|-- poster_path: string (nullable = true)
|-- homepage: string (nullable = true)
|-- imdb_id: string (nullable = true)
|-- genres: string (nullable = true)
|-- video: string (nullable = true)
|-- id: integer (nullable = true)
```

[67]: 43998

```
[68]: # Right Join
df.join(df_p1, df['id'] == df_p1['id'], 'right').printSchema()
df.join(df_p1, df['id'] == df_p1['id'], 'right').count()
```

```
root
|-- id: integer (nullable = true)
|-- budget: float (nullable = true)
|-- popularity: float (nullable = true)
|-- release_date: date (nullable = true)
|-- revenue: float (nullable = true)
```

```

|-- title: string (nullable = true)
|-- mean_popularity: double (nullable = true)
|-- varaiance: double (nullable = true)
|-- release_year: integer (nullable = true)
|-- cast: string (nullable = true)
|-- adult: string (nullable = true)
|-- directors: string (nullable = true)
|-- vote_count: string (nullable = true)
|-- spoken_languages: string (nullable = true)
|-- poster_path: string (nullable = true)
|-- homepage: string (nullable = true)
|-- imdb_id: string (nullable = true)
|-- genres: string (nullable = true)
|-- video: string (nullable = true)
|-- id: integer (nullable = true)

```

[68]: 43783

```

[69]: # Outer Join
df.join(df_p1, df['id'] == df_p1['id'], 'outer').printSchema()
df.join(df_p1, df['id'] == df_p1['id'], 'outer').count()

```

```

root
 |-- id: integer (nullable = true)
 |-- budget: float (nullable = true)
 |-- popularity: float (nullable = true)
 |-- release_date: date (nullable = true)
 |-- revenue: float (nullable = true)
 |-- title: string (nullable = true)
 |-- mean_popularity: double (nullable = true)
 |-- varaiance: double (nullable = true)
 |-- release_year: integer (nullable = true)
 |-- cast: string (nullable = true)
 |-- adult: string (nullable = true)
 |-- directors: string (nullable = true)
 |-- vote_count: string (nullable = true)
 |-- spoken_languages: string (nullable = true)
 |-- poster_path: string (nullable = true)
 |-- homepage: string (nullable = true)
 |-- imdb_id: string (nullable = true)
 |-- genres: string (nullable = true)
 |-- video: string (nullable = true)
 |-- id: integer (nullable = true)

```

[69]: 43998

```
[70]: # Left Anti Join
df.join(df_p1, df['id'] == df_p1['id'], 'left_anti').printSchema()
df.join(df_p1, df['id'] == df_p1['id'], 'left_anti').count()
```

```
root
|-- id: integer (nullable = true)
|-- budget: float (nullable = true)
|-- popularity: float (nullable = true)
|-- release_date: date (nullable = true)
|-- revenue: float (nullable = true)
|-- title: string (nullable = true)
|-- mean_popularity: double (nullable = false)
|-- varaiance: double (nullable = true)
|-- release_year: integer (nullable = true)
```

[70]: 215

```
[71]: # Left Semi Join
df.join(df_p1, df['id'] == df_p1['id'], 'left_semi').printSchema()
df.join(df_p1, df['id'] == df_p1['id'], 'left_semi').count()
```

```
root
|-- id: integer (nullable = true)
|-- budget: float (nullable = true)
|-- popularity: float (nullable = true)
|-- release_date: date (nullable = true)
|-- revenue: float (nullable = true)
|-- title: string (nullable = true)
|-- mean_popularity: double (nullable = false)
|-- varaiance: double (nullable = true)
|-- release_year: integer (nullable = true)
```

[71]: 43783

```
[72]: # Broadcast Join
df.join(broadcast(df_p1), df['id'] == df_p1['id'], 'left_semi').count()
```

[72]: 43783

```
[73]: # Original data count
df.count()
```

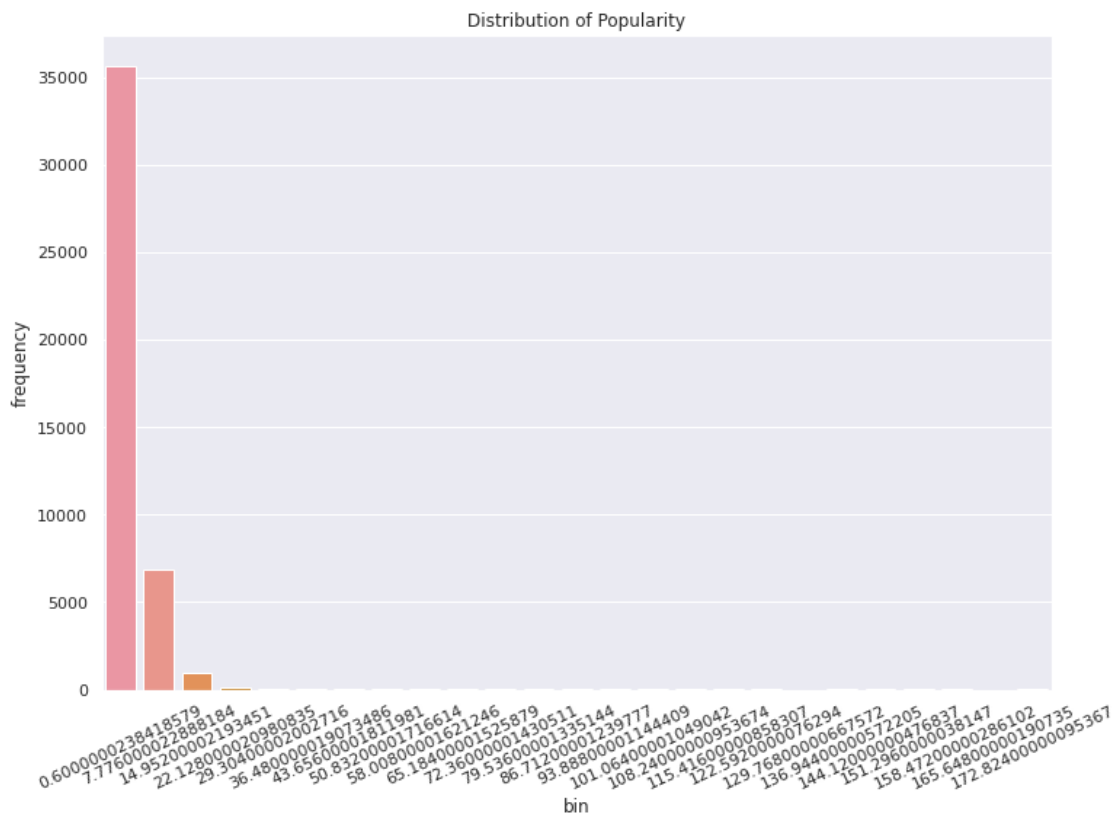
[73]: 43998

```
[74]: # Drop Duplicates Demonstration
df.dropDuplicates(['title','release_year']).count()
```

```
[74]: 43643
```

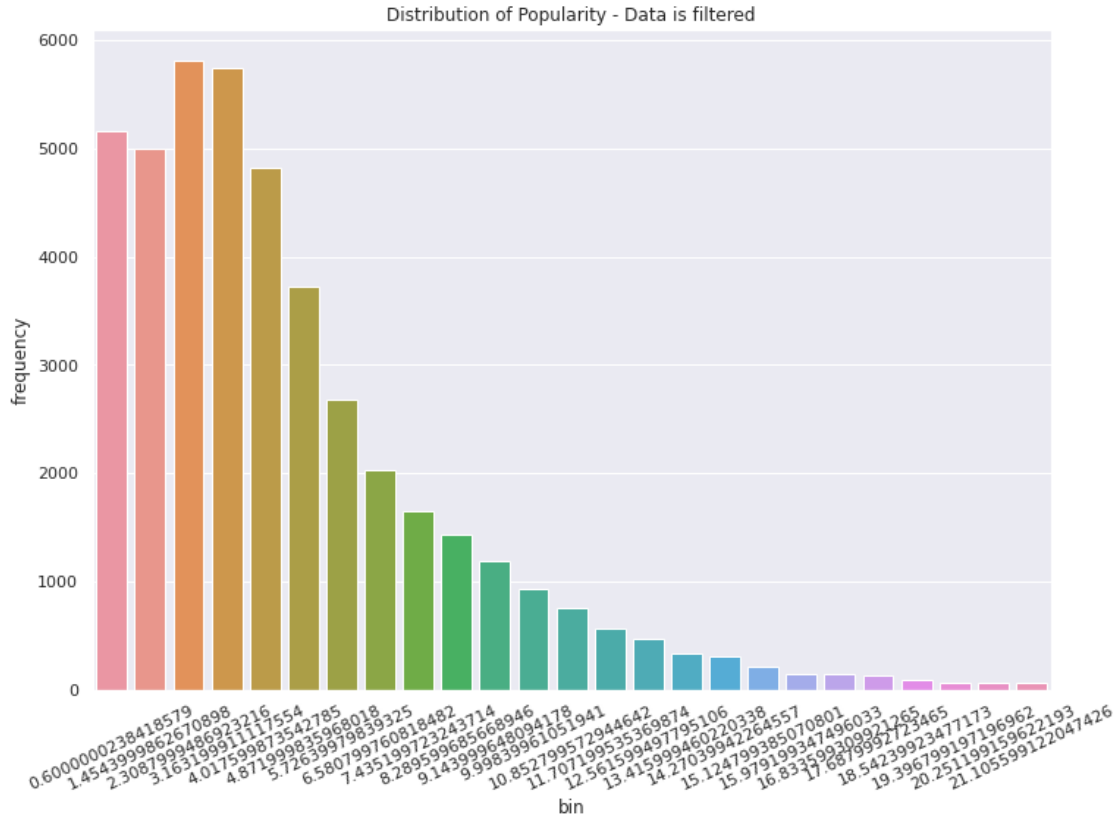
```
[75]: # Histogram Demonstration
%matplotlib inline
import pandas as pd
from matplotlib import pyplot as plt
import seaborn as sns
# Processing the data in Spark. We can use the histogram function from the RDD
histogram_data = df.select('popularity').rdd.flatMap(lambda x: x).histogram(25)
# Loading the Computed Histogram into a Pandas Dataframe for plotting
hist_df=pd.DataFrame(list(zip(*histogram_data)), columns=['bin', 'frequency'])
```

```
[76]: # Plotting
sns.set(rc={'figure.figsize':(11.7,8.27)})
sns.barplot(hist_df['bin'],hist_df['frequency'])
plt.xticks(rotation=25)
plt.title('Distribution of Popularity')
plt.show()
# plt.savefig('myimage.svg', format='svg', dpi=1200)
```



```
[77]: # Filtering the data to get a better understanding of data
df_fil=df.filter('popularity<22')
# Processing the data in Spark. We can use the histogram function from the RDD
histogram_data = df_fil.select('popularity').rdd.flatMap(lambda x: x).
    ↪ histogram(25)
# Loading the Computed Histogram into a Pandas Dataframe for plotting
hist_df=pd.DataFrame(list(zip(*histogram_data)), columns=['bin', 'frequency'])

sns.set(rc={'figure.figsize':(11.7,8.27)})
sns.barplot(hist_df['bin'],hist_df['frequency'])
plt.xticks(rotation=25)
plt.title('Distribution of Popularity - Data is filtered')
plt.show()
# plt.savefig('myimage1.svg', format='svg', dpi=1200)
```



```
[78]: # Data preparation for bar chart
df_cat=df.filter("(release_year>1959) and (release_year<1971)").
    ↪ groupby('release_year').count().toPandas()
df_cat=df_cat.sort_values(by=['release_year'], ascending=False)
```

```
[79]: #Plotting
sns.set(rc={'figure.figsize':(11.7,8.27)})
sns.barplot(df_cat['release_year'],df_cat['count'])
plt.xticks(rotation=25)
plt.title('Number of films released each year from 1960 to 1970 in our dataset')
plt.show()
# plt.savefig('myimage2.svg', format='svg', dpi=1200)
```

